

Politecnico di Torino

Master of Science in Computer Engineering
Department of Control and Computer Engineering (DAUIN)

Large Language Models for Software Engineering

Laboratory 09: LLM-Based Code Generation and Evaluation

Complete notebook-aligned solution

Course Instructor
Prof. Flavio Giobergia

Prepared by
Nicola Beccaceci

Laboratory Assistant
Claudio Savelli

Academic Year
2026/2027

Contents

1 Grade generation **2**

2 Multiple agents **11**

3 Additional functions **13**
3.1 What these comparisons do and do not show 18

1 Exercise 1: generating and evaluating the grade function

The first exercise turns a precise grade rule into an executable reference, checks it at boundaries, and measures two observed generated implementations against the same test suite. The process moves from specification and tests to inference, evaluation, static analysis, and a compact metric plot.

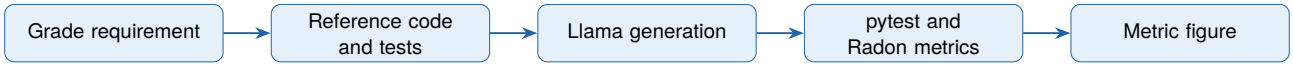


Figure 1: The grade solution is evaluated against a fixed contract and test suite.

The input is six grades and an explicit validity rule; the desired result is a trimmed mean and predictable exceptions. A useful comparison must therefore check both valid numerical results and rejection behavior, then interpret test outcomes alongside static metrics.

We start with an executable reference for the grade requirement: exactly six finite real values, each from 18 through 30 or equal to 33 for laude; one minimum and one maximum are removed, and the input must remain unchanged. The function checks types and values before sorting, so malformed input cannot silently become a plausible average. A fresh Codex-authored baseline is stored with its provenance; it is not presented as a ChatGPT response.

```python
def compute_average(grades):
    """Trim one minimum and maximum from six valid grades; do not mutate input."""
    from numbers import Real
    import math
    if not isinstance(grades, (list, tuple)) or len(grades) != 6:
        raise ValueError("Exactly six grades are required")
    for grade in grades:
        if isinstance(grade, bool):
            raise ValueError("Boolean values are not grades")
        if not isinstance(grade, Real):
            raise TypeError("Grades must be real numbers")
        if not math.isfinite(grade) or not (18 <= grade <= 30 or grade == 33):
            raise ValueError("Grades must be between 18 and 30, or 33 for laude")
    return sum(sorted(grades)[1:-1]) / 4

from pathlib import Path
import json

print(f"The computed average is: {compute_average([18,25,30,33,22,28]):.2f}")
# Fresh code authored by this Codex agent in this session, not a captured ChatGPT UI response.
# Preserve provenance rather than falsely labeling this as an external GPT run.
codex_result = 'def compute_average(grades):\n    """Return the trimmed mean of six grades, with 33 representing laude."""\n
      import math\n    from numbers import Real\n    if not isinstance(grades, (list, tuple)) or len(grades) != 6:\n
     raise ValueError("Expected exactly six grades")\n    for value in grades:\n        if isinstance(value, bool):\n
        raise ValueError("A boolean is not a grade")\n        if not isinstance(value, Real):\n            raise TypeError("
     Expected numeric grades")\n        if not math.isfinite(value) or not (18 <= value <= 30 or value == 33):\n
     raise ValueError("Invalid grade")\n    return (sum(grades) - min(grades) - max(grades)) / 4\n'
my_codes = {"Codex": codex_result}
code_provenance = {"Codex": "Fresh Codex-generated baseline from this solving session"}
Path("function_01.py").write_text(codex_result)
print(code_provenance)

sys_prompt = "You are a Python programmer. Return only valid Python source, without Markdown fences or explanations."
query = """Implement compute_average(grades). Require exactly six grades; accept real values from 18 to 30 inclusive or 33 for
     laude. Reject booleans/invalid values with ValueError and nonnumeric values with TypeError. Return the arithmetic mean
     after removing exactly one minimum and one maximum. Do not mutate the input."""
```

```
--- notebook output 0 ---
The computed average is: 26.25
{'Codex': 'Fresh Codex-generated baseline from this solving session'}
```

The sample `[18, 25, 30, 33, 22, 28]` produces 26.25 after removing 18 and 33 and averaging the remaining four grades. The accompanying provenance dictionary identifies the generated source as a new Codex baseline from this session.

The next cell expresses the contract as black-box pytest tests. Parametrized cases cover ordinary and laude values, tied minima or maxima, fractional grades, invalid list lengths and bounds, NaN/infinity, booleans, and nonnumeric types. It also verifies that valid lists are not mutated. These tests establish the reference oracle before generated implementations are compared.

```python
import pytest

@pytest.mark.parametrize("grades, expected", [
    ([18,25,30,33,22,28],26.25),
```

```
    ([20,25,30,33,29,28],28.0),
    ([18,19,20,30,33,25],23.5),
    ([18]*6,18.0), ([30]*6,30.0), ([33]*6,33.0),
    ([25]*6,25.0), ([18,18,18,33,33,33],25.5),
    ([18.5,20,22,24,26,30],23.0),
])
def test_valid_grades(grades, expected):
    original = grades.copy()
    assert compute_average(grades) == pytest.approx(expected, abs=1e-10)
    assert grades == original

@pytest.mark.parametrize("grades", [[18]*5,[18]*7,[],[17,20,22,24,26,30],
    [34,20,22,24,26,30],[31,20,22,24,26,30],[32,20,22,24,26,30],
    [float("nan"),20,22,24,26,30],[float("inf"),20,22,24,26,30],
    [True,20,22,24,26,30]])
def test_invalid_grades(grades):
    with pytest.raises(ValueError): compute_average(grades)

@pytest.mark.parametrize("value", ["25",None,complex(25,0)])
def test_non_numeric_grades(value):
    with pytest.raises(TypeError): compute_average([value,20,22,24,26,30])
```

This cell defines the test cases but does not execute them; the following cell runs them against the reference function.

The notebook runs the tests in a fresh ipytest module and stops immediately if the reference fails. That guard prevents subsequent model comparisons from measuring a faulty oracle. The run also calls out a disagreement in one unchanged supplied test: its 1

```
import ipytest
ipytest.autoconfig()
# A clean notebook test module, independent of test files used for model comparison.
reference_exit_code = ipytest.run("-vv")
if int(reference_exit_code) != 0:
    raise RuntimeError("Reference tests failed; fix the reference before comparing generated code")
print("All notebook reference boundary/validation tests passed.")
print("Supplied test_valid_grades_with_30laude expects 27.75 with 1% tolerance; exact answer is 28.0.")
print("The supplied test file is preserved; strict tests above check the exact result.")
```

```
--- notebook output 0 ---
============================================ test session starts =============================================
platform darwin -- Python 3.11.16, pytest-9.1.1, pluggy-1.7.0 -- /Users/nicolabeccaceci/.venvs/llms-lab09/bin/python
cachedir: .pytest_cache
rootdir: /Users/nicolabeccaceci/Desktop/LLMs/LABS/lab09
plugins: platformdirs-4.13.5
collecting ...

--- notebook output 1 ---
collected 22 items

t_de183bbd677e46caace029b07ed9ee5e.py::test_valid_grades[grades0-26.25]

--- notebook output 2 ---
PASSED                [  4%]

--- notebook output 3 ---

t_de183bbd677e46caace029b07ed9ee5e.py::test_valid_grades[grades1-28.0]

--- notebook output 4 ---
PASSED                [  9%]

--- notebook output 5 ---

t_de183bbd677e46caace029b07ed9ee5e.py::test_valid_grades[grades2-23.5]

--- notebook output 6 ---
PASSED                [ 13%]

--- notebook output 7 ---

t_de183bbd677e46caace029b07ed9ee5e.py::test_valid_grades[grades3-18.0]

--- notebook output 8 ---
PASSED                [ 18%]

--- notebook output 9 ---

t_de183bbd677e46caace029b07ed9ee5e.py::test_valid_grades[grades4-30.0]

--- notebook output 10 ---
PASSED                [ 22%]

--- notebook output 11 ---

t_de183bbd677e46caace029b07ed9ee5e.py::test_valid_grades[grades5-33.0]
```

```
--- notebook output 12 ---
PASSED               [ 27%]

--- notebook output 13 ---

t_de183bbd677e46caace029b07ed9ee5e.py::test_valid_grades[grades6-25.0]

--- notebook output 14 ---
PASSED               [ 31%]

--- notebook output 15 ---

t_de183bbd677e46caace029b07ed9ee5e.py::test_valid_grades[grades7-25.5]

--- notebook output 16 ---
PASSED               [ 36%]

--- notebook output 17 ---

t_de183bbd677e46caace029b07ed9ee5e.py::test_valid_grades[grades8-23.0]

--- notebook output 18 ---
PASSED               [ 40%]

--- notebook output 19 ---

t_de183bbd677e46caace029b07ed9ee5e.py::test_invalid_grades[grades0]

--- notebook output 20 ---
PASSED                 [ 45%]

--- notebook output 21 ---

t_de183bbd677e46caace029b07ed9ee5e.py::test_invalid_grades[grades1]

--- notebook output 22 ---
PASSED                 [ 50%]

--- notebook output 23 ---

t_de183bbd677e46caace029b07ed9ee5e.py::test_invalid_grades[grades2]

--- notebook output 24 ---
PASSED                 [ 54%]

--- notebook output 25 ---

t_de183bbd677e46caace029b07ed9ee5e.py::test_invalid_grades[grades3]

--- notebook output 26 ---
PASSED                 [ 59%]

--- notebook output 27 ---

t_de183bbd677e46caace029b07ed9ee5e.py::test_invalid_grades[grades4]

--- notebook output 28 ---
PASSED                 [ 63%]

--- notebook output 29 ---

t_de183bbd677e46caace029b07ed9ee5e.py::test_invalid_grades[grades5]

--- notebook output 30 ---
PASSED                 [ 68%]

--- notebook output 31 ---

t_de183bbd677e46caace029b07ed9ee5e.py::test_invalid_grades[grades6]

--- notebook output 32 ---
PASSED                 [ 72%]

--- notebook output 33 ---

t_de183bbd677e46caace029b07ed9ee5e.py::test_invalid_grades[grades7]

--- notebook output 34 ---
PASSED                 [ 77%]

--- notebook output 35 ---

t_de183bbd677e46caace029b07ed9ee5e.py::test_invalid_grades[grades8]

--- notebook output 36 ---
PASSED                 [ 81%]

--- notebook output 37 ---
```

```
t_de183bbd677e46caace029b07ed9ee5e.py::test_invalid_grades[grades9]

--- notebook output 38 ---
PASSED                  [ 86%]

--- notebook output 39 ---

t_de183bbd677e46caace029b07ed9ee5e.py::test_non_numeric_grades[25]

--- notebook output 40 ---
PASSED                   [ 90%]

--- notebook output 41 ---

t_de183bbd677e46caace029b07ed9ee5e.py::test_non_numeric_grades[None]

--- notebook output 42 ---
PASSED                  [ 95%]

--- notebook output 43 ---

t_de183bbd677e46caace029b07ed9ee5e.py::test_non_numeric_grades[(25+0j)]

--- notebook output 44 ---
PASSED               [100%]

--- notebook output 45 ---


======================================== 22 passed in 0.04s ========================================
All notebook reference boundary/validation tests passed.
Supplied test_valid_grades_with_30laude expects 27.75 with 1% tolerance; exact answer is 28.0.
The supplied test file is preserved; strict tests above check the exact result.
```

All 22 notebook-authored reference tests pass. The supplied test is preserved rather than edited; a strict exact-value assertion checks the 28.0 result independently.

With a passing reference, this cell prepares the pinned local inference workflow. It chooses MPS when available, seeds generation, and keeps only one model resident at a time, explicitly clearing accelerator memory before switching. The requested Hugging Face model ID is `meta-llama/Llama-3.2-3B-Instruct`; despite the original notebook wording, it is Llama 3.2, not CodeLlama. The helper returns only generated continuation text, while `apply_chat_template` will serialize system and user messages for the model.

```python
import gc
import torch
from huggingface_hub import get_token
from transformers import AutoTokenizer, AutoModelForCausalLM, set_seed

set_seed(42)
torch.set_num_threads(2)
device = "cuda" if torch.cuda.is_available() else "mps" if torch.backends.mps.is_available() else "cpu"
print("Using device:", device)
model = tokenizer = None
current_model_id = None

def select_engine(model_id):
    """Hold one model at a time so comparisons fit the local memory budget."""
    global model, tokenizer, current_model_id
    if current_model_id == model_id and model is not None:
        return
    model = tokenizer = None
    current_model_id = None
    gc.collect()
    if device == "mps": torch.mps.empty_cache()
    if device == "cuda": torch.cuda.empty_cache()
    dtype = torch.float16 if device != "cpu" else torch.float32
    model = AutoModelForCausalLM.from_pretrained(
        model_id, torch_dtype=dtype, low_cpu_mem_usage=True,
        token=get_token(),
    ).to(device).eval()
    tokenizer = AutoTokenizer.from_pretrained(model_id, token=get_token())
    tokenizer.pad_token = tokenizer.eos_token
    current_model_id = model_id

def make_a_query(prompt, tokenizer, model, device, max_new_tokens=768):
    inputs = tokenizer(prompt, return_tensors="pt", add_special_tokens=False).to(device)
    with torch.inference_mode():
        output = model.generate(**inputs, max_new_tokens=max_new_tokens,
            do_sample=False, pad_token_id=tokenizer.eos_token_id)
    return tokenizer.decode(output[0, inputs["input_ids"].shape[1]:], skip_special_tokens=True).strip()

# The supplied checkpoint is Llama 3.2, despite the original heading saying CodeLlama.
model_id = "meta-llama/Llama-3.2-3B-Instruct"
select_engine(model_id)
```

```
--- notebook output 0 ---
/Users/nicolabeccaceci/.venvs/llms-lab09/lib/python3.11/site-packages/tqdm/auto.py:21: TqdmWarning: IProgress not found. Please update
      jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm

--- notebook output 1 ---
Using device: mps

--- notebook output 2 ---

Loading checkpoint shards:   0%|          | 0/2 [00:00<?, ?it/s]

--- notebook output 3 ---

Loading checkpoint shards:  50%|#####     | 1/2 [00:12<00:12, 12.47s/it]

--- notebook output 4 ---

Loading checkpoint shards: 100%|##########| 2/2 [00:16<00:00,  7.30s/it]

--- notebook output 5 ---

Loading checkpoint shards: 100%|##########| 2/2 [00:16<00:00,  8.07s/it]

--- notebook output 6 ---
```

The clean execution selected MPS and loaded both checkpoint shards. The widget progress warning is environmental display noise; the weights loaded successfully. The notebook does not pin a repository commit, so the PDF records the model ID and runtime environment without inventing a revision.

Now the grade requirements are framed as a system instruction and a user task, then serialized with the model's official chat template and an assistant-generation prefix. Greedy decoding requests at most 768 new tokens. The code removes only an enclosing Markdown fence if one appears and stores the actual result and model ID for downstream test execution.

```
sys_prompt = "You are a Python programmer. Return only valid Python source, without Markdown fences or explanations."
query = """Implement compute_average(grades). Require exactly six grades; accept real values from 18 to 30 inclusive or 33 for
      laude. Reject booleans/invalid values with ValueError and nonnumeric values with TypeError. Return the arithmetic mean
      after removing exactly one minimum and one maximum. Do not mutate the input."""
prompt = tokenizer.apply_chat_template([
    {"role":"system","content":sys_prompt},
    {"role":"user","content":query},
], tokenize=False, add_generation_prompt=True)
generated_code = make_a_query(prompt, tokenizer, model, device)
# Remove only an enclosing Markdown code fence if the instruction was not followed.
if generated_code.startswith("‘‘‘"):
    generated_code = generated_code.split("\n",1)[1].rsplit("‘‘‘",1)[0].strip()
my_codes["Llama-3.2-3B-Instruct"] = generated_code
code_provenance["Llama-3.2-3B-Instruct"] = current_model_id
print(generated_code)
```

```
--- notebook output 0 ---
/Users/nicolabeccaceci/.venvs/llms-lab09/lib/python3.11/site-packages/transformers/generation/configuration_utils.py:590: UserWarning:
      ‘do_sample‘ is set to ‘False‘. However, ‘temperature‘ is set to ‘0.6‘ -- this flag is only used in sample-based generation modes
     . You should set ‘do_sample=True‘ or unset ‘temperature‘.
  warnings.warn(
/Users/nicolabeccaceci/.venvs/llms-lab09/lib/python3.11/site-packages/transformers/generation/configuration_utils.py:595: UserWarning:
      ‘do_sample‘ is set to ‘False‘. However, ‘top_p‘ is set to ‘0.9‘ -- this flag is only used in sample-based generation modes. You
     should set ‘do_sample=True‘ or unset ‘top_p‘.
  warnings.warn(

--- notebook output 1 ---
def compute_average(grades):
    """
    Compute the arithmetic mean of a list of grades after removing the minimum and maximum values.

    Args:
        grades (list): A list of grades. Each grade should be a real number between 18 and 30 inclusive or 33 for laude.

    Returns:
        float: The arithmetic mean of the remaining grades.

    Raises:
        ValueError: If the list contains booleans, invalid values, or nonnumeric values.
        TypeError: If the input is not a list.
    """

    # Check if input is a list
    if not isinstance(grades, list):
        raise TypeError("Input must be a list")

    # Check if list contains booleans or invalid values
    if not all(isinstance(grade, (int, float)) and 18 <= grade <= 30 or grade == 33 for grade in grades):
        raise ValueError("List must contain only real numbers between 18 and 30 inclusive or 33 for laude")
```

```
    # Create a copy of the list to avoid mutation
    grades_copy = grades.copy()

    # Remove the minimum and maximum values
    grades_copy.remove(min(grades_copy))
    grades_copy.remove(max(grades_copy))

    # Check if list is empty after removing minimum and maximum values
    if len(grades_copy) == 0:
        raise ValueError("List must contain at least two values")

    # Compute the arithmetic mean
    average = sum(grades_copy) / len(grades_copy)

    return average
```

The recorded Llama completion is printed in full in the output block. The generation configuration also emits warnings because temperature and top-p are set while `do_sample=False`; those sampling controls are inert under greedy decoding. The generated function is tested next without rewriting or repairing it.

To evaluate generated code, this helper runs pytest in a temporary directory with a fresh import path, writes JUnit XML, and parses individual test outcomes instead of relying on fragile summary-text parsing. It saves stdout and the report under `execution/evaluations/`, then returns passed, failed, error, skipped, and functional-correctness values. This isolates each candidate from stale bytecode and earlier models' files.

```
import hashlib
import re
import shutil
import subprocess
import sys
import tempfile
import xml.etree.ElementTree as ET
from pathlib import Path

def parse_test_results(result_string):
    def count(pattern):
        match = re.search(pattern, result_string)
        return int(match.group(1)) if match else 0
    return count(r"(\d+)\s+errors?"), count(r"(\d+)\s+failed"), count(r"(\d+)\s+passed")

def evaluate_code(code, test_file="test_cases_01.py", function_file="function_01.py", name="generated"):
    # Each subprocess gets a fresh import directory; no model can inherit the prior code's bytecode.
    evidence = Path("execution") / "evaluations" / re.sub(r"[^A-Za-z0-9_.-]", "_", name)
    evidence.mkdir(parents=True, exist_ok=True)
    with tempfile.TemporaryDirectory(prefix="lab09-tests-") as temporary:
        run_dir = Path(temporary)
        (run_dir/function_file).write_text(code)
        shutil.copy2(test_file, run_dir/Path(test_file).name)
        result = subprocess.run([sys.executable,"-m","pytest",Path(test_file).name,
            "-q","--disable-warnings","--tb=short","--color=no","--junitxml=report.xml"],
            cwd=run_dir, capture_output=True, text=True, timeout=60)
        (evidence/"stdout.txt").write_text(result.stdout+result.stderr)
        report = run_dir/"report.xml"
        if not report.exists(): raise RuntimeError("pytest produced no test report: "+result.stderr)
        shutil.copy2(report,evidence/"junit.xml")
        cases = ET.parse(report).getroot().findall(".//testcase")
        failures = sum(case.find("failure") is not None for case in cases)
        errors = sum(case.find("error") is not None for case in cases)
        skipped = sum(case.find("skipped") is not None for case in cases)
        passes = len(cases)-failures-errors-skipped
        total = passes+failures+errors
        if total == 0: raise RuntimeError("No tests ran")
        metrics = {"Passed":passes,"Failed":failures,"Errors":errors,"Skipped":skipped,
            "FunctionalCorrectness":passes/total,"pytest_exit_code":result.returncode,
            "SourceSHA256":hashlib.sha256(code.encode()).hexdigest()}
        (evidence/"metrics.json").write_text(json.dumps(metrics,indent=2))
        print(name,metrics)
        return metrics

codex_metrics = evaluate_code(my_codes["Codex"],name="Codex_grades")
codex_functional_correctness = codex_metrics["FunctionalCorrectness"]
```

```
--- notebook output 0 ---
huggingface/tokenizers: The current process just got forked, after parallelism has already been used. Disabling parallelism to avoid
      deadlocks...
To disable this warning, you can either:
        - Avoid using ‘tokenizers‘ before the fork if possible
        - Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true | false)

--- notebook output 1 ---
Codex_grades {’Passed’: 14, ’Failed’: 0, ’Errors’: 0, ’Skipped’: 0, ’FunctionalCorrectness’: 1.0, ’pytest_exit_code’: 0, ’SourceSHA256
      ’: ’42d51761595f164133f1f4ef76c696cfab648dae72a55d2c2d8b3121030b8462’}
```

The separately authored Codex baseline passes 14 of 14 supplied tests, yielding functional correctness 1.0. The tokenizers-after-fork messages come from subprocess evaluation and do not alter the tests.

The Llama source is now passed through that same evaluator against the same preserved grade tests. The cell stores the observed functional-correctness fraction for the later comparison; it does not infer success from the source looking plausible.

```
llama_label = "Llama-3.2-3B-Instruct"
llama_metrics = evaluate_code(my_codes[llama_label],name="Llama_grades")
llama_functional_correctness = llama_metrics["FunctionalCorrectness"]
```

```
--- notebook output 0 ---
Llama_grades {'Passed': 9, 'Failed': 5, 'Errors': 0, 'Skipped': 0, 'FunctionalCorrectness': 0.6428571428571429, 'pytest_exit_code': 1,
      'SourceSHA256': '71df77699c59cf835031a0116601738837086882f18d4b502d6fab131bffe79a'}

--- notebook output 1 ---
huggingface/tokenizers: The current process just got forked, after parallelism has already been used. Disabling parallelism to avoid
     deadlocks...
To disable this warning, you can either:
        - Avoid using `tokenizers` before the fork if possible
        - Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true | false)
```

Llama passes 9 of 14 grade tests and fails 5, so the measured functional correctness is $9/14 = 0.6429$. The nonzero pytest exit is retained as a real model result, not hidden by the later metrics.

This cell combines static and dynamic evaluation. Radon measures cyclomatic complexity, maintainability index, lines of code, and comments; pytest measures behavior on the shared suite. Each actual source is written to a provenance-named file before analysis. Only sources present in `my_codes` are compared, avoiding a fabricated GPT or ChatGPT result.

```
import radon.complexity as radon_complexity
import radon.metrics as radon_metrics
import radon.raw as radon_raw

def analyze_code_with_radon(file_path):
    code = Path(file_path).read_text()
    functions = radon_complexity.cc_visit(code)
    raw = radon_raw.analyze(code)
    return {"CC":max((block.complexity for block in functions),default=0),
        "MI":radon_metrics.mi_visit(code,multi=False),"LOC":raw.loc,
        "Comments":raw.comments}

def compare_codes(codes, test_file="test_cases_01.py", function_file="function_01.py", suffix="grades"):
    results = {}
    for name, code in codes.items():
        # Export each exact observed source, then measure the same source.
        source_path = Path("execution")/"sources"/(re.sub(r"[^A-Za-z0-9_.-]","_",name)+"_"+suffix+".py")
        source_path.parent.mkdir(parents=True,exist_ok=True)
        source_path.write_text(code)
        static = analyze_code_with_radon(source_path)
        dynamic = evaluate_code(code,test_file,function_file,name+"_"+suffix)
        results[name] = {**static,**dynamic}
    return results

overall_results = compare_codes(my_codes)
print(json.dumps(overall_results,indent=2))
print("Only actually generated sources are included:",code_provenance)
```

```
--- notebook output 0 ---
huggingface/tokenizers: The current process just got forked, after parallelism has already been used. Disabling parallelism to avoid
     deadlocks...
To disable this warning, you can either:
        - Avoid using `tokenizers` before the fork if possible
        - Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true | false)
huggingface/tokenizers: The current process just got forked, after parallelism has already been used. Disabling parallelism to avoid
     deadlocks...
To disable this warning, you can either:
        - Avoid using `tokenizers` before the fork if possible
        - Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true | false)

--- notebook output 1 ---
Codex_grades {'Passed': 14, 'Failed': 0, 'Errors': 0, 'Skipped': 0, 'FunctionalCorrectness': 1.0, 'pytest_exit_code': 0, 'SourceSHA256
     ': '42d51761595f164133f1f4ef76c696cfab648dae72a55d2c2d8b3121030b8462'}

--- notebook output 2 ---
Llama-3.2-3B-Instruct_grades {'Passed': 9, 'Failed': 5, 'Errors': 0, 'Skipped': 0, 'FunctionalCorrectness': 0.6428571428571429, '
     pytest_exit_code': 1, 'SourceSHA256': '71df77699c59cf835031a0116601738837086882f18d4b502d6fab131bffe79a'}
{
  "Codex": {
    "CC": 9,
    "MI": 58.00073622123373,
    "LOC": 14,
    "Comments": 0,
```

```
    "Passed": 14,
    "Failed": 0,
    "Errors": 0,
    "Skipped": 0,
    "FunctionalCorrectness": 1.0,
    "pytest_exit_code": 0,
    "SourceSHA256": "42d51761595f164133f1f4ef76c696cfab648dae72a55d2c2d8b3121030b8462"
  },
  "Llama-3.2-3B-Instruct": {
    "CC": 7,
    "MI": 89.72051787423352,
    "LOC": 38,
    "Comments": 6,
    "Passed": 9,
    "Failed": 5,
    "Errors": 0,
    "Skipped": 0,
    "FunctionalCorrectness": 0.6428571428571429,
    "pytest_exit_code": 1,
    "SourceSHA256": "71df77699c59cf835031a0116601738837086882f18d4b502d6fab131bffe79a"
  }
}
Only actually generated sources are included: {'Codex': 'Fresh Codex-generated baseline from this solving session', 'Llama-3.2-3B-
     Instruct': 'meta-llama/Llama-3.2-3B-Instruct'}
```

For grades, Codex records CC 9, MI 58.00, 14 LOC, and 0 comments, with 14/14 tests passing. Llama records CC 7, MI 89.72, 38 LOC, and 6 comments, with 9/14 passing. A higher maintainability score in this single run does not compensate for failures or establish an overall quality ranking.

The metrics are plotted on separate axes because they use different scales: complexity, maintainability, lines, comments, and test pass fraction are not directly commensurate quantities. The plot gives a compact view of the two observed grade implementations; it is a visualization of this run, not a statistical comparison across repeated samples.

```python
import matplotlib.pyplot as plt
import numpy as np


languages = list(overall_results.keys())



cc_values = [overall_results[language]['CC'] for language in languages]
mi_values = [overall_results[language]['MI'] for language in languages]
loc_values = [overall_results[language]['LOC'] for language in languages]
comments_values = [overall_results[language]['Comments'] for language in languages]
functional_correctness = [overall_results[language]['FunctionalCorrectness'] for language in languages]

# Create a figure with subplots for each metric
fig, axs = plt.subplots(3, 2, figsize=(10, 8))

# Plotting CC values
axs[0, 0].bar(languages, cc_values, color='skyblue')
axs[0, 0].set_title('Cyclomatic Complexity (CC)')
axs[0, 0].set_ylabel('CC')

# Plotting MI values
axs[0, 1].bar(languages, mi_values, color='lightgreen')
axs[0, 1].set_title('Maintainability Index (MI)')
axs[0, 1].set_ylabel('MI')

# Plotting LOC values
axs[1, 0].bar(languages, loc_values, color='blue')
axs[1, 0].set_title('Lines of Code (LOC)')
axs[1, 0].set_ylabel('LOC')

# Plotting Comments values
axs[1, 1].bar(languages, comments_values, color='lightcoral')
axs[1, 1].set_title('Number of Comments')
axs[1, 1].set_ylabel('Comments')
axs[1, 1].set_ylim(0, max(comments_values + [1]) * 1.1)

#plotting functional correctness
axs[2, 0].bar(languages, functional_correctness, color='green')
axs[2, 0].set_title('Functional correctness')
axs[2, 0].set_ylabel('Passed / executed tests')
axs[2, 0].set_ylim(0, 1.05)
axs[2, 1].axis('off')

# Adjust layout and display the plot
plt.tight_layout()
Path("execution/figures").mkdir(parents=True,exist_ok=True)
fig.savefig("execution/figures/grade_code_metrics.png",dpi=160,bbox_inches="tight")
plt.show()
```

```
--- notebook output 0 ---
<Figure size 1000x800 with 6 Axes>
```

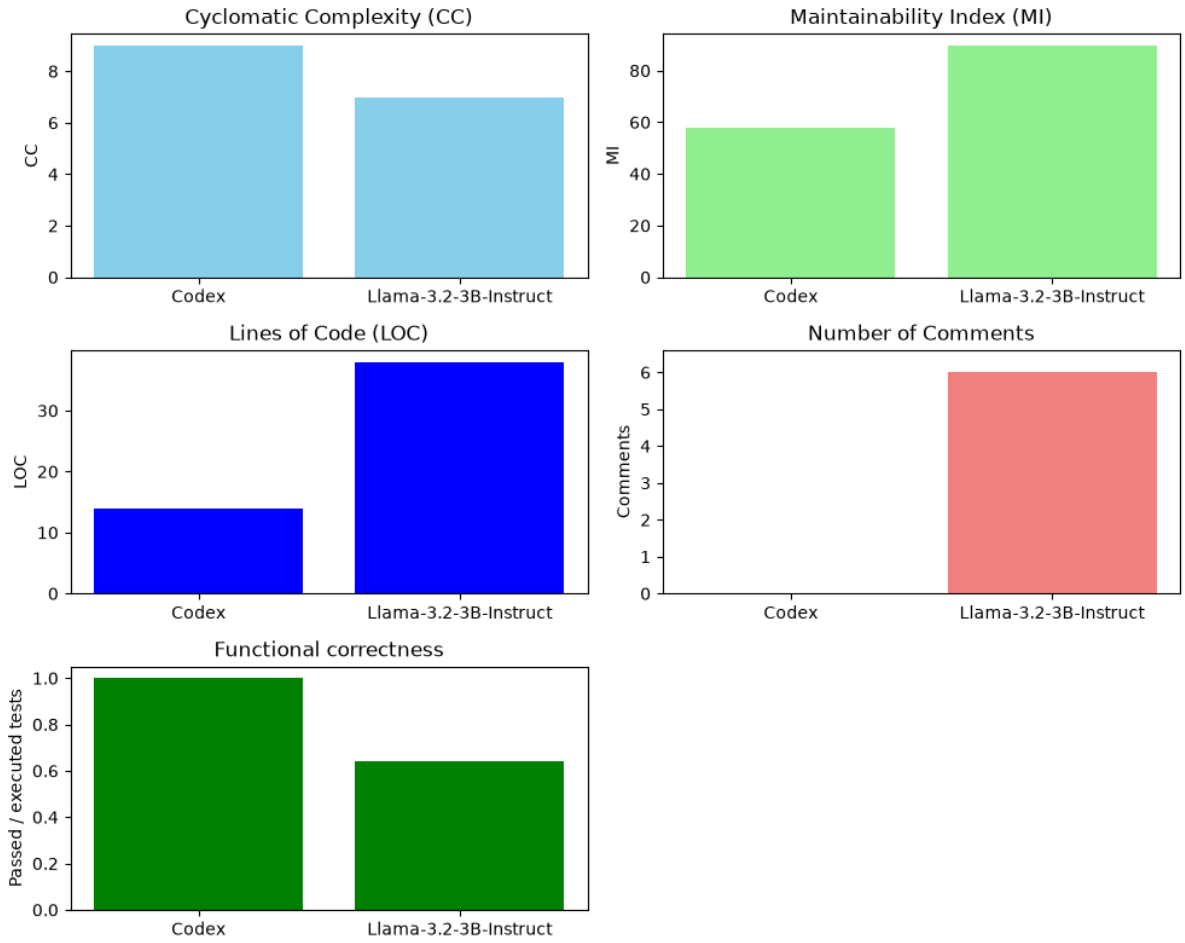


Figure 2: Recorded grade-code comparison: complexity, maintainability, size, comments, and functional correctness.

The saved figure reproduces the cell's five metric panels. The coverage inventory also retains the notebook's original figure payload, and the rendered version is included at readable resolution.

2 Exercise 2: comparing additional code-generation agents

The second exercise keeps the same grade requirement and test oracle while adding the public Qwen 2.5 Coder model. This isolates the comparison to the generated implementation and measured outcome for this task; no external ChatGPT response is claimed.

The second exercise adds the explicitly requested Qwen comparison while keeping the original Llama result and Codex baseline. `select_engine` releases Llama before loading `Qwen/Qwen2.5-Coder-1.5B-Instruct`; the same task prompt, greedy decoding helper, and grading test suite are reused. The additional model is a supplementary local comparison and does not replace the prescribed Llama run.

```
# A separate additional-engine comparison, not a replacement for the specified Llama model.
# Run this when a heavy-compute slot is available; it downloads the named public checkpoint.
additional_model_id = "Qwen/Qwen2.5-Coder-1.5B-Instruct"
select_engine(additional_model_id)
additional_prompt = tokenizer.apply_chat_template([
    {"role":"system","content":sys_prompt},{"role":"user","content":query},
], tokenize=False, add_generation_prompt=True)
qwen_chat = make_a_query(additional_prompt,tokenizer,model,device)
if qwen_chat.startswith("‘‘‘"):
    qwen_chat = qwen_chat.split("\n",1)[1].rsplit("‘‘‘",1)[0].strip()
my_codes["Qwen2.5-Coder-1.5B-Instruct"] = qwen_chat
code_provenance["Qwen2.5-Coder-1.5B-Instruct"] = additional_model_id
print(qwen_chat)
overall_results = compare_codes(my_codes)
print(json.dumps(overall_results,indent=2))
```

```
--- notebook output 0 ---
/Users/nicolabeccaceci/.venvs/llms-lab09/lib/python3.11/site-packages/transformers/generation/configuration_utils.py:590: UserWarning:
      ‘do_sample‘ is set to ‘False‘. However, ‘temperature‘ is set to ‘0.7‘ -- this flag is only used in sample-based generation modes
     . You should set ‘do_sample=True‘ or unset ‘temperature‘.
  warnings.warn(
/Users/nicolabeccaceci/.venvs/llms-lab09/lib/python3.11/site-packages/transformers/generation/configuration_utils.py:595: UserWarning:
      ‘do_sample‘ is set to ‘False‘. However, ‘top_p‘ is set to ‘0.8‘ -- this flag is only used in sample-based generation modes. You
     should set ‘do_sample=True‘ or unset ‘top_p‘.
  warnings.warn(
/Users/nicolabeccaceci/.venvs/llms-lab09/lib/python3.11/site-packages/transformers/generation/configuration_utils.py:612: UserWarning:
      ‘do_sample‘ is set to ‘False‘. However, ‘top_k‘ is set to ‘20‘ -- this flag is only used in sample-based generation modes. You
     should set ‘do_sample=True‘ or unset ‘top_k‘.
  warnings.warn(

--- notebook output 1 ---
def compute_average(grades):
    if len(grades) != 6:
        raise ValueError("Exactly six grades are required.")

    grades = [grade for grade in grades if isinstance(grade, (int, float)) and 18 <= grade <= 30 or grade == 33]
    if len(grades) < 5:
        raise ValueError("At least five grades are required after filtering.")

    min_val = min(grades)
    max_val = max(grades)
    grades.remove(min_val)
    grades.remove(max_val)

    return sum(grades) / len(grades)

--- notebook output 2 ---
huggingface/tokenizers: The current process just got forked, after parallelism has already been used. Disabling parallelism to avoid
     deadlocks...
To disable this warning, you can either:
        - Avoid using ‘tokenizers‘ before the fork if possible
        - Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true | false)

--- notebook output 3 ---
Codex_grades {’Passed’: 14, ’Failed’: 0, ’Errors’: 0, ’Skipped’: 0, ’FunctionalCorrectness’: 1.0, ’pytest_exit_code’: 0, ’SourceSHA256
     ’: ’42d51761595f164133f1f4ef76c696cfab648dae72a55d2c2d8b3121030b8462’}

--- notebook output 4 ---
huggingface/tokenizers: The current process just got forked, after parallelism has already been used. Disabling parallelism to avoid
     deadlocks...
To disable this warning, you can either:
        - Avoid using ‘tokenizers‘ before the fork if possible
        - Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true | false)

--- notebook output 5 ---
Llama-3.2-3B-Instruct_grades {’Passed’: 9, ’Failed’: 5, ’Errors’: 0, ’Skipped’: 0, ’FunctionalCorrectness’: 0.6428571428571429, ’
     pytest_exit_code’: 1, ’SourceSHA256’: ’71df77699c59cf835031a0116601738837086882f18d4b502d6fab131bffe79a’}

--- notebook output 6 ---
Qwen2.5-Coder-1.5B-Instruct_grades {’Passed’: 8, ’Failed’: 6, ’Errors’: 0, ’Skipped’: 0, ’FunctionalCorrectness’: 0.5714285714285714, ’
     pytest_exit_code’: 1, ’SourceSHA256’: ’41c70d817e176461bff36f9832cc21ab5a592899a3461532fd320a4f32ac2ce4’}
{
  "Codex": {
    "CC": 9,
```

```
    "MI": 58.00073622123373,
    "LOC": 14,
    "Comments": 0,
    "Passed": 14,
    "Failed": 0,
    "Errors": 0,
    "Skipped": 0,
    "FunctionalCorrectness": 1.0,
    "pytest_exit_code": 0,
    "SourceSHA256": "42d51761595f164133f1f4ef76c696cfab648dae72a55d2c2d8b3121030b8462"
  },
  "Llama-3.2-3B-Instruct": {
    "CC": 7,
    "MI": 89.72051787423352,
    "LOC": 38,
    "Comments": 6,
    "Passed": 9,
    "Failed": 5,
    "Errors": 0,
    "Skipped": 0,
    "FunctionalCorrectness": 0.6428571428571429,
    "pytest_exit_code": 1,
    "SourceSHA256": "71df77699c59cf835031a0116601738837086882f18d4b502d6fab131bffe79a"
  },
  "Qwen2.5-Coder-1.5B-Instruct": {
    "CC": 7,
    "MI": 62.30659087147264,
    "LOC": 14,
    "Comments": 0,
    "Passed": 8,
    "Failed": 6,
    "Errors": 0,
    "Skipped": 0,
    "FunctionalCorrectness": 0.5714285714285714,
    "pytest_exit_code": 1,
    "SourceSHA256": "41c70d817e176461bff36f9832cc21ab5a592899a3461532fd320a4f32ac2ce4"
  }
}

--- notebook output 7 ---
huggingface/tokenizers: The current process just got forked, after parallelism has already been used. Disabling parallelism to avoid
      deadlocks...
To disable this warning, you can either:
        - Avoid using ‘tokenizers‘ before the fork if possible
        - Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true | false)
```


Qwen passes 8/14 grade tests (0.5714). Static metrics are CC 7, MI 62.31, 14 LOC, and 0 comments; Llama is 9/14 and Codex is 14/14. The full generated Qwen code, warnings, JSON metrics, and recorded model provenance appear in the output. These scores describe this prompt, test set, checkpoints, and single run only.

3 Exercise 3: extending the comparison to railway and bike tasks

The final exercise changes the specification to two different branching problems. A railway discount depends on group composition and size, while the bike task maps speed bands and track types to a maximum time. The same source-generation, test-execution, and static-analysis pipeline is reused so the comparison includes actual observed code and outcomes for each task.

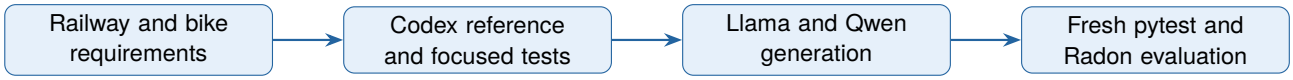


Figure 3: The additional tasks reuse the same validated comparison pipeline.

The third exercise introduces two more requirements: a group railway fee and a track-dependent maximum time. This cell creates a fresh Codex implementation and focused black-box tests for each task, writes the source and test files, and validates the local baselines before generating any further model candidates. The fee rule gives a discount only when a group has at least two passengers and an adult; the bike rule has exact speed boundaries for track types A and B and a fixed multiplier for C.

```
# Fresh Codex-generated source and black-box tests for the two additional requirements.
# These are genuine local agent outputs; Llama/Qwen comparison requires actual generation below.
fee_source = 'def computeFee(basePrice, n_passengers, n_over18, n_under15):\n    from numbers import Real\n    import math\n    if isinstance(basePrice,bool) or not isinstance(basePrice,Real) or not math.isfinite(basePrice) or basePrice < 0:\n        raise ValueError("Invalid base price")\n    counts = (n_passengers,n_over18,n_under15)\n    if any(isinstance(n,bool) or not isinstance(n,int) for n in counts):\n        raise ValueError("Passenger counts must be integers")\n    if not 1 <= n_passengers <= 5 or min(n_over18,n_under15) < 0 or n_over18+n_under15 > n_passengers:\n        raise ValueError("Inconsistent group counts")\n    eligible = n_passengers >= 2 and n_over18 >= 1\n    return float(basePrice * (n_passengers-n_under15 if eligible else n_passengers))\n'
bike_source = 'def computeMaxTime(winner_time, avg_speed, track_type):\n    import math\n    from numbers import Real\n    if any(isinstance(v,bool) or not isinstance(v,Real) or not math.isfinite(v) or v <= 0 for v in (winner_time,avg_speed)):\n        return 0\n    if track_type not in ("A","B","C"):\n        return 0\n    if track_type == "C":\n        return winner_time * 1.5\n    band = 0 if avg_speed <= 30 else 1 if avg_speed <= 35 else 2\n    increment = {"A":(0.05,0.10,0.15),"B":(0.20,0.25,0.30)}[track_type][band]\n    return winner_time * (1 + increment)\n'
Path("function_02.py").write_text(fee_source)
Path("function_03.py").write_text(bike_source)
Path("test_cases_02.py").write_text('import pytest\nfrom function_02 import computeFee\n@pytest.mark.parametrize("args,expected", [((10,2,1,1),10),((10,5,1,4),10),((10,5,1,0),50),((10,1,1,0),10),((10,2,0,2),20),((0,3,1,2),0)])\ndef test_fee(args,expected): assert computeFee(*args) == pytest.approx(expected)\n@pytest.mark.parametrize("args", [(10,6,1,2),(10,0,0,0),(10,3,2,2),(10,3,-1,1),(-1,2,1,1),(10,2.5,1,1),(10,True,0,0)])\ndef test_invalid_fee(args):\n    with pytest.raises(ValueError): computeFee(*args)\n')
Path("test_cases_03.py").write_text('import pytest\nfrom function_03 import computeMaxTime\n@pytest.mark.parametrize("category,speed,expected", [("A",30,105),("A",30.001,110),("A",35,110),("A",35.001,115),("B",30,120),("B",30.001,125),("B",35,125),("B",35.001,130),("C",20,150),("C",40,150)])\ndef test_boundaries(category,speed,expected): assert computeMaxTime(100,speed,category) == pytest.approx(expected)\n@pytest.mark.parametrize("args", [(0,30,"A"),(-1,30,"A"),(100,0,"A"),(100,-1,"A"),(100,30,"D"),(100,30,"a"),(float("nan"),30,"A"),(100,True,"A"),("100",30,"A")])\ndef test_invalid(args): assert computeMaxTime(*args) == 0\n')
additional_tasks = {
 "railway": {"function_file":"function_02.py","test_file":"test_cases_02.py","sources":{"Codex":fee_source},
 "prompt":"Implement computeFee(basePrice,n_passengers,n_over18,n_under15). Groups of 2 to 5 with at least one adult aged 18+ pay only for members aged 15+. Others pay basePrice each. Reject invalid counts, groups larger than 5, negative price, counts inconsistent with group size. Ages under15 and at least18 are disjoint. Raise ValueError for invalid inputs."},
 "bike": {"function_file":"function_03.py","test_file":"test_cases_03.py","sources":{"Codex":bike_source},
 "prompt":"Implement computeMaxTime(winner_time,avg_speed,track_type). Positive finite numeric time and speed required; invalid input returns 0. Track A increases winner time by 5% at speed<=30,10% at 30<speed<=35,15% at speed>35. B uses20%,25%,30% on the same intervals. C always adds50%. Valid track types are uppercase A,B,C."},
}
additional_results = {}
for task_name, task in additional_tasks.items():
    additional_results[task_name] = compare_codes(task["sources"],task["test_file"],task["function_file"],task_name)
print(json.dumps(additional_results,indent=2))
```

```
--- notebook output 0 ---
huggingface/tokenizers: The current process just got forked, after parallelism has already been used. Disabling parallelism to avoid deadlocks...
To disable this warning, you can either:
        - Avoid using `tokenizers` before the fork if possible
        - Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true | false)

--- notebook output 1 ---
Codex_railway {'Passed': 13, 'Failed': 0, 'Errors': 0, 'Skipped': 0, 'FunctionalCorrectness': 1.0, 'pytest_exit_code': 0, 'SourceSHA256': 'e9f6a232bfeb302e7537f2a9a1acad1fee1e2b8191ed0f0d2f42735e73f3b34e'}

--- notebook output 2 ---
Codex_bike {'Passed': 19, 'Failed': 0, 'Errors': 0, 'Skipped': 0, 'FunctionalCorrectness': 1.0, 'pytest_exit_code': 0, 'SourceSHA256': 'afb17cd190a85eeafcdef8771f27df4eebfd52cc37a85b288dc8f8bcd0d77dcb'}
{
  "railway": {
    "Codex": {
      "CC": 13,
```

```
      "MI": 57.69893707064606,
      "LOC": 12,
      "Comments": 0,
      "Passed": 13,
      "Failed": 0,
      "Errors": 0,
      "Skipped": 0,
      "FunctionalCorrectness": 1.0,
      "pytest_exit_code": 0,
      "SourceSHA256": "e9f6a232bfeb302e7537f2a9a1acad1fee1e2b8191ed0f0d2f42735e73f3b34e"
    }
  },
  "bike": {
    "Codex": {
      "CC": 10,
      "MI": 59.01647343290267,
      "LOC": 12,
      "Comments": 0,
      "Passed": 19,
      "Failed": 0,
      "Errors": 0,
      "Skipped": 0,
      "FunctionalCorrectness": 1.0,
      "pytest_exit_code": 0,
      "SourceSHA256": "afb17cd190a85eeafcdef8771f27df4eebfd52cc37a85b288dc8f8bcd0d77dcb"
    }
  }
}

--- notebook output 3 ---
huggingface/tokenizers: The current process just got forked, after parallelism has already been used. Disabling parallelism to avoid
      deadlocks...
To disable this warning, you can either:
        - Avoid using ‘tokenizers‘ before the fork if possible
        - Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true | false)
```


The two Codex baselines pass their observed suites: 13/13 railway cases and 19/19 bike cases. The source strings and test files become the comparison state consumed by the final cell.

Finally, the workflow loops over Llama and Qwen sequentially and over both new tasks. Each candidate is generated from the matching task prompt, saved as the observed source, and evaluated with the same test and Radon pipeline. The loop prints candidate code and a JSON result object for every engine-task pair; it never loads both checkpoints together.

```
# Adapt the same actual generation/test/static-analysis workflow to the two new tasks.
# Checkpoints are sequentially released by select_engine, never loaded together.
for engine_label, engine_id in [("Llama-3.2-3B-Instruct","meta-llama/Llama-3.2-3B-Instruct"),
                                ("Qwen2.5-Coder-1.5B-Instruct",additional_model_id)]:
    select_engine(engine_id)
    for task_name, task in additional_tasks.items():
        prompt = tokenizer.apply_chat_template([
            {"role":"system","content":sys_prompt},{"role":"user","content":task["prompt"]}],
            tokenize=False,add_generation_prompt=True)
        code = make_a_query(prompt,tokenizer,model,device)
        if code.startswith("‘‘‘"): code = code.split("\n",1)[1].rsplit("‘‘‘",1)[0].strip()
        print(engine_label,task_name,code)
        task["sources"][engine_label] = code
        additional_results[task_name] = compare_codes(task["sources"],task["test_file"],task["function_file"],task_name)
print(json.dumps(additional_results,indent=2))
```


```
--- notebook output 0 ---

Loading checkpoint shards:   0%|          | 0/2 [00:00<?, ?it/s]

--- notebook output 1 ---

Loading checkpoint shards:  50%|#####     | 1/2 [00:10<00:10, 10.42s/it]

--- notebook output 2 ---

Loading checkpoint shards: 100%|##########| 2/2 [00:14<00:00,  6.53s/it]

--- notebook output 3 ---

Loading checkpoint shards: 100%|##########| 2/2 [00:14<00:00,  7.11s/it]

--- notebook output 4 ---


--- notebook output 5 ---
/Users/nicolabeccaceci/.venvs/llms-lab09/lib/python3.11/site-packages/transformers/generation/configuration_utils.py:590: UserWarning:
      ‘do_sample‘ is set to ‘False‘. However, ‘temperature‘ is set to ‘0.6‘ -- this flag is only used in sample-based generation modes
     . You should set ‘do_sample=True‘ or unset ‘temperature‘.
  warnings.warn(
/Users/nicolabeccaceci/.venvs/llms-lab09/lib/python3.11/site-packages/transformers/generation/configuration_utils.py:595: UserWarning:
      ‘do_sample‘ is set to ‘False‘. However, ‘top_p‘ is set to ‘0.9‘ -- this flag is only used in sample-based generation modes. You
     should set ‘do_sample=True‘ or unset ‘top_p‘.
```

```
  warnings.warn(

--- notebook output 6 ---
Llama-3.2-3B-Instruct railway def computeFee(basePrice, n_passengers, n_over18, n_under15):
    """
    Compute the fee for a group of passengers.

    Args:
    basePrice (float): The base price per passenger.
    n_passengers (int): The total number of passengers.
    n_over18 (int): The number of passengers aged 18 or older.
    n_under15 (int): The number of passengers under 15.

    Returns:
    float: The total fee for the group.

    Raises:
    ValueError: If the inputs are invalid.
    """

    # Check if the inputs are valid
    if not isinstance(basePrice, (int, float)) or basePrice < 0:
        raise ValueError("Invalid base price")
    if not isinstance(n_passengers, int) or n_passengers <= 0:
        raise ValueError("Invalid number of passengers")
    if not isinstance(n_over18, int) or n_over18 < 0:
        raise ValueError("Invalid number of passengers over 18")
    if not isinstance(n_under15, int) or n_under15 < 0:
        raise ValueError("Invalid number of passengers under 15")

    # Check if the counts are consistent with the group size
    if n_passengers < 2 or n_passengers > 5:
        raise ValueError("Invalid group size")
    if n_over18 + n_under15 > n_passengers:
        raise ValueError("Invalid counts")

    # Compute the fee
    fee = 0
    for i in range(2, 6):
        # Calculate the number of groups of i passengers
        groups = n_passengers // i
        # Calculate the number of passengers in each group
        passengers_in_group = n_passengers % i
        # Calculate the fee for each group
        group_fee = basePrice * i
        # Add the fee for the current group to the total fee
        fee += group_fee * groups
        # Add the fee for the remaining passengers to the total fee
        fee += basePrice * passengers_in_group

    # Subtract the fee for passengers under 15 from the total fee
    fee -= n_under15 * basePrice

    return fee

--- notebook output 7 ---
huggingface/tokenizers: The current process just got forked, after parallelism has already been used. Disabling parallelism to avoid
      deadlocks...
To disable this warning, you can either:
        - Avoid using ‘tokenizers‘ before the fork if possible
        - Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true | false)

--- notebook output 8 ---
Codex_railway {’Passed’: 13, ’Failed’: 0, ’Errors’: 0, ’Skipped’: 0, ’FunctionalCorrectness’: 1.0, ’pytest_exit_code’: 0, ’
      SourceSHA256’: ’e9f6a232bfeb302e7537f2a9a1acad1fee1e2b8191ed0f0d2f42735e73f3b34e’}
Llama-3.2-3B-Instruct_railway {’Passed’: 8, ’Failed’: 5, ’Errors’: 0, ’Skipped’: 0, ’FunctionalCorrectness’: 0.6153846153846154, ’
      pytest_exit_code’: 1, ’SourceSHA256’: ’b9a1dfa0a90811cc69044a556f33dd79300d15a7436991950abb9dd19af47e45’}

--- notebook output 9 ---
huggingface/tokenizers: The current process just got forked, after parallelism has already been used. Disabling parallelism to avoid
      deadlocks...
To disable this warning, you can either:
        - Avoid using ‘tokenizers‘ before the fork if possible
        - Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true | false)

--- notebook output 10 ---
Llama-3.2-3B-Instruct bike def computeMaxTime(winner_time, avg_speed, track_type):
    """
    Compute the maximum time a winner can have on a track.

    Args:
    winner_time (float): The initial time of the winner.
    avg_speed (float): The average speed of the winner.
    track_type (str): The type of the track. Can be ’A’, ’B’, or ’C’.

    Returns:
    float: The maximum time the winner can have on the track.
    """

    # Check if inputs are valid
```

```
    if not isinstance(winner_time, (int, float)) or winner_time <= 0:
        return 0
    if not isinstance(avg_speed, (int, float)) or avg_speed <= 0:
        return 0
    if not isinstance(track_type, str) or len(track_type)!= 1 or not track_type.isupper():
        return 0

    # Define the track types and their corresponding time multipliers
    track_types = {
        'A': {'speed': 30,'multiplier': 0.05},
        'B': {'speed': 35,'multiplier': 0.20},
        'C': {'speed': float('inf'),'multiplier': 0.50}
    }

    # Check if the track type is valid
    if track_type not in track_types:
        return 0

    # Calculate the multiplier based on the speed
    if avg_speed <= track_types[track_type]['speed']:
        multiplier = track_types[track_type]['multiplier']
    else:
        multiplier = 0

    # Calculate the maximum time
    max_time = winner_time * (1 + multiplier)

    return max_time

--- notebook output 11 ---
huggingface/tokenizers: The current process just got forked, after parallelism has already been used. Disabling parallelism to avoid
      deadlocks...
To disable this warning, you can either:
        - Avoid using ‘tokenizers‘ before the fork if possible
        - Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true | false)

--- notebook output 12 ---
Codex_bike {'Passed': 19, 'Failed': 0, 'Errors': 0, 'Skipped': 0, 'FunctionalCorrectness': 1.0, 'pytest_exit_code': 0, 'SourceSHA256':
       'afb17cd190a85eeafcdef8771f27df4eebfd52cc37a85b288dc8f8bcd0d77dcb'}

--- notebook output 13 ---
huggingface/tokenizers: The current process just got forked, after parallelism has already been used. Disabling parallelism to avoid
      deadlocks...
To disable this warning, you can either:
        - Avoid using ‘tokenizers‘ before the fork if possible
        - Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true | false)

--- notebook output 14 ---
Llama-3.2-3B-Instruct_bike {'Passed': 11, 'Failed': 8, 'Errors': 0, 'Skipped': 0, 'FunctionalCorrectness': 0.5789473684210527, '
      pytest_exit_code': 1, 'SourceSHA256': '46eb38bb921c9c8d7d5b5369f63add95f221177508bfd403950c840d81b0ba67'}

--- notebook output 15 ---
Qwen2.5-Coder-1.5B-Instruct railway def compute_fee(basePrice, n_passengers, n_over18, n_under15):
    if not (0 <= basePrice <= 100) or not (0 <= n_passengers <= 100) or not (0 <= n_over18 <= 100) or not (0 <= n_under15 <= 100):
        raise ValueError("Invalid input values")

    if n_passengers > 5:
        raise ValueError("Group size cannot be more than 5")

    if n_over18 + n_under15 != n_passengers:
        raise ValueError("Counts must match the number of passengers")

    if n_under15 >= n_over18:
        return basePrice * n_passengers

    return basePrice * n_over18 + (basePrice / 2) * n_under15

--- notebook output 16 ---
huggingface/tokenizers: The current process just got forked, after parallelism has already been used. Disabling parallelism to avoid
      deadlocks...
To disable this warning, you can either:
        - Avoid using ‘tokenizers‘ before the fork if possible
        - Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true | false)

--- notebook output 17 ---
Codex_railway {'Passed': 13, 'Failed': 0, 'Errors': 0, 'Skipped': 0, 'FunctionalCorrectness': 1.0, 'pytest_exit_code': 0, '
      SourceSHA256': 'e9f6a232bfeb302e7537f2a9a1acad1fee1e2b8191ed0f0d2f42735e73f3b34e'}

--- notebook output 18 ---
huggingface/tokenizers: The current process just got forked, after parallelism has already been used. Disabling parallelism to avoid
      deadlocks...
To disable this warning, you can either:
        - Avoid using ‘tokenizers‘ before the fork if possible
        - Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true | false)

--- notebook output 19 ---
Llama-3.2-3B-Instruct_railway {'Passed': 8, 'Failed': 5, 'Errors': 0, 'Skipped': 0, 'FunctionalCorrectness': 0.6153846153846154, '
      pytest_exit_code': 1, 'SourceSHA256': 'b9a1dfa0a90811cc69044a556f33dd79300d15a7436991950abb9dd19af47e45'}

--- notebook output 20 ---
```

```
huggingface/tokenizers: The current process just got forked, after parallelism has already been used. Disabling parallelism to avoid
      deadlocks...
To disable this warning, you can either:
        - Avoid using ‘tokenizers‘ before the fork if possible
        - Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true | false)

--- notebook output 21 ---
Qwen2.5-Coder-1.5B-Instruct_railway {’Passed’: 0, ’Failed’: 0, ’Errors’: 1, ’Skipped’: 0, ’FunctionalCorrectness’: 0.0, ’
      pytest_exit_code’: 2, ’SourceSHA256’: ’02d35fe69e71af1eb973447b507a24ffad4082b5a462bab6d3058e35e2a23dea’}

--- notebook output 22 ---
Qwen2.5-Coder-1.5B-Instruct bike def computeMaxTime(winner_time, avg_speed, track_type):
    if not isinstance(winner_time, (int, float)) or winner_time <= 0:
        return 0
    if not isinstance(avg_speed, (int, float)) or avg_speed <= 0:
        return 0
    if track_type not in [’A’, ’B’, ’C’]:
        return 0

    if track_type == ’A’:
        if avg_speed <= 30:
            return winner_time * 1.05
        elif avg_speed <= 35:
            return winner_time * 1.10
        else:
            return winner_time * 1.15
    elif track_type == ’B’:
        if avg_speed <= 30:
            return winner_time * 1.20
        elif avg_speed <= 35:
            return winner_time * 1.25
        else:
            return winner_time * 1.30
    else:  # track_type == ’C’
        return winner_time * 1.50

--- notebook output 23 ---
huggingface/tokenizers: The current process just got forked, after parallelism has already been used. Disabling parallelism to avoid
      deadlocks...
To disable this warning, you can either:
        - Avoid using ‘tokenizers‘ before the fork if possible
        - Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true | false)

--- notebook output 24 ---
Codex_bike {’Passed’: 19, ’Failed’: 0, ’Errors’: 0, ’Skipped’: 0, ’FunctionalCorrectness’: 1.0, ’pytest_exit_code’: 0, ’SourceSHA256’:
       ’afb17cd190a85eeafcdef8771f27df4eebfd52cc37a85b288dc8f8bcd0d77dcb’}

--- notebook output 25 ---
huggingface/tokenizers: The current process just got forked, after parallelism has already been used. Disabling parallelism to avoid
      deadlocks...
To disable this warning, you can either:
        - Avoid using ‘tokenizers‘ before the fork if possible
        - Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true | false)

--- notebook output 26 ---
Llama-3.2-3B-Instruct_bike {’Passed’: 11, ’Failed’: 8, ’Errors’: 0, ’Skipped’: 0, ’FunctionalCorrectness’: 0.5789473684210527, ’
      pytest_exit_code’: 1, ’SourceSHA256’: ’46eb38bb921c9c8d7d5b5369f63add95f221177508bfd403950c840d81b0ba67’}

--- notebook output 27 ---
Qwen2.5-Coder-1.5B-Instruct_bike {’Passed’: 17, ’Failed’: 2, ’Errors’: 0, ’Skipped’: 0, ’FunctionalCorrectness’: 0.8947368421052632, ’
      pytest_exit_code’: 1, ’SourceSHA256’: ’d03228a1692906b3aa497b88a68ce5eadc3df693592198e6da93c10058c8681e’}
{
  "railway": {
    "Codex": {
      "CC": 13,
      "MI": 57.69893707064606,
      "LOC": 12,
      "Comments": 0,
      "Passed": 13,
      "Failed": 0,
      "Errors": 0,
      "Skipped": 0,
      "FunctionalCorrectness": 1.0,
      "pytest_exit_code": 0,
      "SourceSHA256": "e9f6a232bfeb302e7537f2a9a1acad1fee1e2b8191ed0f0d2f42735e73f3b34e"
    },
    "Llama-3.2-3B-Instruct": {
      "CC": 13,
      "MI": 78.73755739125758,
      "LOC": 51,
      "Comments": 9,
      "Passed": 8,
      "Failed": 5,
      "Errors": 0,
      "Skipped": 0,
      "FunctionalCorrectness": 0.6153846153846154,
      "pytest_exit_code": 1,
      "SourceSHA256": "b9a1dfa0a90811cc69044a556f33dd79300d15a7436991950abb9dd19af47e45"
    },
    "Qwen2.5-Coder-1.5B-Instruct": {
```

```
      "CC": 8,
      "MI": 59.85280151682359,
      "LOC": 14,
      "Comments": 0,
      "Passed": 0,
      "Failed": 0,
      "Errors": 1,
      "Skipped": 0,
      "FunctionalCorrectness": 0.0,
      "pytest_exit_code": 2,
      "SourceSHA256": "02d35fe69e71af1eb973447b507a24ffad4082b5a462bab6d3058e35e2a23dea"
    }
  },
  "bike": {
    "Codex": {
      "CC": 10,
      "MI": 59.01647343290267,
      "LOC": 12,
      "Comments": 0,
      "Passed": 19,
      "Failed": 0,
      "Errors": 0,
      "Skipped": 0,
      "FunctionalCorrectness": 1.0,
      "pytest_exit_code": 0,
      "SourceSHA256": "afb17cd190a85eeafcdef8771f27df4eebfd52cc37a85b288dc8f8bcd0d77dcb"
    },
    "Llama-3.2-3B-Instruct": {
      "CC": 10,
      "MI": 80.51480541410928,
      "LOC": 42,
      "Comments": 5,
      "Passed": 11,
      "Failed": 8,
      "Errors": 0,
      "Skipped": 0,
      "FunctionalCorrectness": 0.5789473684210527,
      "pytest_exit_code": 1,
      "SourceSHA256": "46eb38bb921c9c8d7d5b5369f63add95f221177508bfd403950c840d81b0ba67"
    },
    "Qwen2.5-Coder-1.5B-Instruct": {
      "CC": 12,
      "MI": 63.802904183778594,
      "LOC": 24,
      "Comments": 1,
      "Passed": 17,
      "Failed": 2,
      "Errors": 0,
      "Skipped": 0,
      "FunctionalCorrectness": 0.8947368421052632,
      "pytest_exit_code": 1,
      "SourceSHA256": "d03228a1692906b3aa497b88a68ce5eadc3df693592198e6da93c10058c8681e"
    }
  }
}

--- notebook output 28 ---
huggingface/tokenizers: The current process just got forked, after parallelism has already been used. Disabling parallelism to avoid
      deadlocks...
To disable this warning, you can either:
        - Avoid using `tokenizers` before the fork if possible
        - Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true | false)
```


For railway, Llama passes 8/13 tests (0.6154); Qwen's generated source yields one collection/runtime error before any tests run, so its 0.0 metric is not equivalent to a tested 0

3.1 What these comparisons do and do not show

The reference and fresh Codex baseline pass the observed grade, railway, and bike tests. Llama and Qwen results vary by task, and one Qwen railway run raises a test-collection error rather than producing a measured set of passes and failures. Static metrics such as lines of code and cyclomatic complexity describe the source's structure; they do not guarantee that it follows the requirements. Each pass fraction is tied to a specific finite suite, prompt, checkpoint ID, and local run.

The Lab 09 lecture recording was not provided for this notebook-only pass. This solution makes no lecture-derived claims; its explanations and results come from the assignment notebook, local model execution, and saved test artifacts.